JavaScript is disabled on your browser.

compact1, compact2, compact3

java.util.function

Interface BiConsumer<T,U>

Type Parameters:

`T - the type of the first argument to the operation`

`U - the type of the second argument to the operation`

Functional Interface:

`This is a functional interface and can therefore be used as the assignment target for a lambda expression or method reference.`

```
@FunctionalInterface
public interface BiConsumer<T,U>
```

Represents an operation that accepts two input arguments and returns no result. This is the two-arity specialization of `Consumer`. Unlike most other functional interfaces, `BiConsumer` is expected to operate via side-effects.

This is a functional interface whose functional method is `accept(Object, Object)`.

Since:

1.8

See Also:

`Consumer`

Method Summary

Abstract Methods | Default Methods

Modifier and Type	Method	Description

void	**accept**(**T** t, **U** u)	Performs this operation on the given arguments.
default **BiConsumer**<**T**,**U**>	**andThen**(**BiConsumer**<? super **T**,? super **U**> after)	Returns a composed `BiConsumer` that performs, in sequence, this operation followed by the `after` operation.

Method Detail

accept

```
void accept(T t,
            U u)
```

Performs this operation on the given arguments.

Parameters:

`t - the first input argument`

`u - the second input argument`

andThen

```
default BiConsumer<T,U> andThen(BiConsumer<? super T,? super U> after)
```

Returns a composed `BiConsumer` that performs, in sequence, this operation followed by the `after` operation. If performing either operation throws an exception, it is relayed to the caller of the composed operation. If performing this operation throws an exception, the `after` operation will not be performed.

Parameters:

`after - the operation to perform after this operation`

Returns:

`a composed BiConsumer that performs in sequence this operation followed by the after operation`

Throws:

`NullPointerException - if after is null`

Submit a bug or feature
For further API reference and developer documentation, see Java SE Documentation. That documentation contains more detailed, developer-targeted descriptions, with conceptual overviews, definitions of terms, workarounds, and working code examples.
Copyright © 1993, 2026, Oracle and/or its affiliates. All rights reserved. Use is subject to license terms.
Also see the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.